

INTERNSHIP PROJECT REPORT

DESIGN AND IMPLEMENTATION OF A 16-BIT PROCESSOR USING VERILOG HDL

Submitted by:

POOJA RAJEEVAYYA KABBINAKANTIMATH

JUNE 2026

ABSTRACT

This project presents the design and implementation of a 16-bit processor using Verilog HDL. The processor is composed of several modules including an Arithmetic Logic Unit (ALU), Control Unit, Register File, Instruction Decoder, Program Counter, and RAM. The processor executes instructions through a sequence of operations such as instruction fetch, decode, register read, execute, register write-back, and memory access. Simulation and verification were performed using Verilog testbenches to ensure correct functionality of all modules.

1. INTRODUCTION

A processor is the central component of a digital system responsible for executing instructions and processing data. This project demonstrates the design of a simple 16-bit processor architecture using Verilog HDL. The processor supports arithmetic, logical, shift, compare, load immediate, and branching operations.

The complete design consists of:

- Arithmetic Logic Unit (ALU)
- Control Unit
- Instruction Decoder (processor16)
- Register File
- Program Counter
- Fake RAM
- Top-Level Integration Module
- Verification Testbenches

2. OBJECTIVES

1. Design a 16-bit processor using Verilog HDL.
2. Implement arithmetic and logical operations.
3. Design a register file with multiple registers.
4. Implement instruction decoding and execution.
5. Verify processor functionality through simulation.

3. PROCESSOR ARCHITECTURE

The processor follows a modular architecture.

Modules Used:

1. alu.v
2. ctrl_unit.v

3. processor16.v
4. register_file.v
5. pc_unit.v
6. fake_ram.v
7. main_test.v
8. register_unitest.v
9. decoder_unitest.v

Data Flow:

Program Counter → RAM → Decoder → Register File → ALU → Register File

The Control Unit coordinates the execution of all stages.

4. MODULE DESCRIPTION

4.1 Arithmetic Logic Unit (alu.v)

The ALU performs all arithmetic and logical operations.

Supported Operations:

- ADD
- SUB
- OR
- AND
- XOR
- NOT
- LOAD IMMEDIATE
- COMPARE
- SHIFT LEFT
- SHIFT RIGHT
- JUMP ABSOLUTE
- JUMP RELATIVE

Inputs:

- Clock
- Enable
- ALU Opcode
- Data A

- Data B
- Immediate Data

Outputs:

- ALU Result
- Branch Control Signal

Verilog code:

```
`timescale 1ns / 1ps
```

```
module alu(
    input    I_clk,
    input    I_en,
    input [4:0] I_aluop,
    input [15:0] I_dataA,
    input [15:0] I_dataB,
    input [7:0] I_imm,
    output reg [15:0] O_dataResult,
    output reg    O_shldBranch
);
    reg [17:0] int_result;
    wire op_lsb;
    wire [3:0] opcode;
    wire [15:0] aluData;
    localparam Add    = 4'd0,
               Sub    = 4'd1,
               OR_OP  = 4'd2,
               AND_OP = 4'd3,
               XOR_OP = 4'd4,
               NOT_OP = 4'd5,
               Rdmem  = 4'd6,
               Wrmem  = 4'd7,
               Load   = 4'd8,
               Cmp    = 4'd9,
               SHL    = 4'd10,
```

```

        SHR    = 4'd11,
        JMPA   = 4'd12,
        JMPR   = 4'd13;
assign op_lsb = I_aluop[0];
assign opcode = I_aluop[4:1];
initial begin
    int_result  = 18'd0;
    O_dataResult = 16'd0;
    O_shldBranch = 1'b0;
end
always @(negedge I_clk)
begin
    if(I_en)
    begin
        O_shldBranch <= 1'b0;
        case(opcode)
            // ADD
            Add:
            begin
                int_result <= (op_lsb) ?
                ($signed(I_dataA) + $signed(I_dataB)) :
                (I_dataA + I_dataB);
            end
            // SUB
            Sub:
            begin
                int_result <= (op_lsb) ?
                ($signed(I_dataA) - $signed(I_dataB)) :
                (I_dataA - I_dataB);
            end
            // OR
            OR_OP:

```

```

begin
    int_result <= I_dataA | I_dataB;
end
// AND
AND_OP:
begin
    int_result <= I_dataA & I_dataB;
end
// XOR
XOR_OP:
begin
    int_result <= I_dataA ^ I_dataB;
end
// NOT
NOT_OP:
begin
int_result <= ~I_dataA;
end
// LOAD IMMEDIATE
Load:
begin
    if(op_lsb)
        int_result <= {I_imm, 8'h00};
    else
        int_result <= {8'h00, I_imm};
    end
// COMPARE
Cmp:
begin
    int_result <= 18'd0;
    if(op_lsb)
begin

```

```

        int_result[0] <= ($signed(I_dataA) == $signed(I_dataB));
        int_result[1] <= ($signed(I_dataA) == 0);
        int_result[2] <= ($signed(I_dataB) == 0);
        int_result[3] <= ($signed(I_dataA) > $signed(I_dataB));
        int_result[4] <= ($signed(I_dataA) < $signed(I_dataB));
    end
else
begin
    int_result[0] <= (I_dataA == I_dataB);
    int_result[1] <= (I_dataA == 16'd0);
    int_result[2] <= (I_dataB == 16'd0);
    int_result[3] <= (I_dataA > I_dataB);
    int_result[4] <= (I_dataA < I_dataB);
end
end

// SHIFT LEFT

SHL:
begin
    int_result <= I_dataA << I_dataB[3:0];
end

// SHIFT RIGHT

SHR:
begin
    int_result <= I_dataA >> I_dataB[3:0];
end

// JUMP ABSOLUTE

JMPA:
begin
    int_result <= (op_lsb) ? I_dataA : {8'h00, I_imm};
    O_shldBranch <= 1'b1;
end

// JUMP RELATIVE

```

```

JMPR:
begin
    int_result <= I_dataA;
    case({op_lsb, I_imm[1:0]})
        3'b000: O_shldBranch <= I_dataB[0];
        3'b001: O_shldBranch <= I_dataB[1];
        3'b010: O_shldBranch <= I_dataB[2];
        3'b011: O_shldBranch <= I_dataB[3];
        default: O_shldBranch <= 1'b0;
    endcase
end
default:
begin
    int_result <= 18'd0;
end
endcase
O_dataResult <= int_result[15:0];
end
end
endmodule

```

4.2 Control Unit (ctrl_unit.v)

The Control Unit is implemented using a finite state machine.

Execution States:

1. Fetch
2. Decode
3. Register Read
4. Execute
5. Register Write
6. Memory Access

The control unit generates enable signals for each stage of processor execution.

Verilog code:


```

`timescale 1ns / 1ps

// Module Definition
module ctrl_unit(
    // Inputs
    input I_clk,
    input I_reset,
    // Outputs
    output O_enfetch,
    output O_endec,
    output O_enrgrd,
    output O_enalu,
    output O_enrgwr,
    output O_enmem
);
// Register Declaration
reg [5:0] state;
// Initial Block
initial begin
    state <= 6'b000001;
end
// State Select Block
always @(posedge I_clk) begin
    if (I_reset)
        state <= 6'b000001;
    else begin
        case(state)
            6'b000001 : state <= 6'b000010;
            6'b000010 : state <= 6'b000100;
            6'b000100 : state <= 6'b001000;
            6'b001000 : state <= 6'b010000;
            6'b010000 : state <= 6'b100000;
            6'b100000 : state <= 6'b000001;
        endcase
    end
end

```

```

        default : state <= 6'b0000001;
    endcase
end
end

// Assignment Enable Signals
assign O_enfetch = state[0];
assign O_endec  = state[1];
assign O_enrgrd = state[2];
assign O_enalu  = state[3];
assign O_energwr = state[4];
assign O_enmem  = state[5];
endmodule

```

4.3 Register File (register_file.v)

The register file contains sixteen 16-bit registers.

Features:

- 16 General Purpose Registers
- Dual Read Ports
- Single Write Port

Operations:

- Read Register A
- Read Register B
- Write Destination Register

Verilog code:

```

`timescale 1ns / 1ps

module register_file(
    // Inputs
    input    I_clk,
    input    I_en,
    input    I_we,
    input [3:0] I_selA,
    input [3:0] I_selB,

```

```

input [3:0] I_selD,
input [15:0] I_dataD,
// Outputs
output [15:0] O_dataA,
output [15:0] O_dataB
);

// Register Bank : R0 to R15
reg [15:0] r [0:15];
integer i;
// Initialize Registers
initial
begin
    for(i = 0; i < 16; i = i + 1)
        r[i] = 16'h0000;
    end

// Write Operation
always @(posedge I_clk)
begin
    if(I_en && I_we)
        r[I_selD] <= I_dataD;
    end

// Read Operations
assign O_dataA = (I_en) ? r[I_selA] : 16'h0000;
assign O_dataB = (I_en) ? r[I_selB] : 16'h0000;
endmodule

```

4.4 Instruction Decoder (processor16.v)

The instruction decoder extracts fields from the 16-bit instruction.

Instruction Format:

- Bits [15:11] → ALU Opcode
- Bits [10:8] → Source Register A
- Bits [7:5] → Source Register B

- Bits [4:2] → Destination Register
- Bits [7:0] → Immediate Data

The decoder also generates the register write enable signal.

Verilog code:

```
`timescale 1ns / 1ps

module processor16(
    // Inputs
    input    I_clk,
    input    I_en,
    input [15:0] I_inst,
    // Outputs
    output reg [4:0] O_aluop,
    output reg [3:0] O_selA,
    output reg [3:0] O_selB,
    output reg [3:0] O_selD,
    output reg [7:0] O_imm,
    output reg    O_regwe
);

// Initialize Outputs
initial begin
    O_aluop = 5'b00000;
    O_selA  = 4'b0000;
    O_selB  = 4'b0000;
    O_selD  = 4'b0000;
    O_imm   = 8'b00000000;
    O_regwe = 1'b0;
end

// Instruction Decode
always @(negedge I_clk)
begin
    if(I_en)
        begin
```

```

// Instruction Format
// [15:11] -> ALU Opcode
// [10:8] -> Source Register A
// [7:5] -> Source Register B
// [4:2] -> Destination Register
// [7:0] -> Immediate Data
O_aluop <= I_inst[15:11];
O_selA <= {1'b0, I_inst[10:8]};
O_selB <= {1'b0, I_inst[7:5]};
O_selD <= {1'b0, I_inst[4:2]};
O_imm <= I_inst[7:0];
case(I_inst[15:12])
    // STORE
    4'b0111 :
        O_regwe <= 1'b0;
    // JUMP ABSOLUTE
    4'b1100 :
        O_regwe <= 1'b0;
    // JUMP RELATIVE
    4'b1101 :
        O_regwe <= 1'b0;
    // All other instructions write back
    default :
        O_regwe <= 1'b1;
endcase
end
else
begin
    O_regwe <= 1'b0;
end
end
end

```

endmodule

4.5 Program Counter (pc_unit.v)

The Program Counter stores the address of the next instruction.

Functions:

- Hold Current Address
- Increment Address
- Load New Address
- Reset

Verilog Code:

```
`timescale 1ns / 1ps
```

```
module pc_unit(
```

```
    // Inputs
```

```
    input    I_clk,
```

```
    input [1:0] I_opcode,
```

```
    input [15:0] I_pc,
```

```
    // Output
```

```
    output reg [15:0] O_pc
```

```
);
```

```
    initial
```

```
    begin
```

```
        O_pc = 16'd0;
```

```
    end
```

```
    always @(negedge I_clk)
```

```
    begin
```

```
        case(I_opcode)
```

```
            2'b00 : O_pc <= O_pc;    // Hold
```

```
            2'b01 : O_pc <= O_pc + 1; // Increment
```

```
            2'b10 : O_pc <= I_pc;    // Load New PC
```

```
            2'b11 : O_pc <= 16'd0;  // Reset
```

```
        endcase
```

```
    end
```

```
endmodule
```

4.6 Fake RAM (fake_ram.v)

The RAM module stores instructions and data.

Functions:

- Read Data
- Write Data
- Store Program Instructions

The memory is initialized with sample instructions for processor execution.

Verilog code:

```
`timescale 1ns / 1ps
```

```
module fake_ram(
```

```
    // Inputs
```

```
    input    I_clk,
```

```
    input    I_we,
```

```
    input [15:0] I_addr,
```

```
    input [15:0] I_data,
```

```
    // Output
```

```
    output reg [15:0] O_data
```

```
);
```

```
    // Memory: 9 locations (0-8)
```

```
    reg [15:0] mem [0:8];
```

```
    initial
```

```
    begin
```

```
        mem[0] = 16'b1000000011111110;
```

```
        mem[1] = 16'b1000100111101101;
```

```
        mem[2] = 16'b0010001000100000;
```

```
        mem[3] = 16'b1000001100000001;
```

```
        mem[4] = 16'b1000010000000001;
```

```
        mem[5] = 16'b0000001101110000;
```

```
        mem[6] = 16'b1100000000000101;
```

```
        mem[7] = 16'b0000000000000000;
```

```
        mem[8] = 16'b0000000000000000;
```

```
        O_data = 16'b0000000000000000;
```

```

end
always @(negedge I_clk)
begin
    if(I_we)
        mem[I_addr[3:0]] <= I_data;
        O_data <= mem[I_addr[3:0]];
    end
endmodule

```

4.7 Top-Level Module (main_test.v)

The top-level module integrates all processor components.

Integrated Modules:

- Register File
- Instruction Decoder
- ALU
- Control Unit
- Program Counter
- RAM

The module verifies complete processor functionality.

Verilog code:

```

`timescale 1ns / 1ps

module main_test;

    reg clk;
    reg reset;
    wire regwe;
    wire [3:0] selA;
    wire [3:0] selB;
    wire [3:0] selD;
    wire [15:0] dataA;
    wire [15:0] dataB;
    wire [15:0] aluData;
    wire [15:0] dataO;

```



```

wire [4:0] aluop;
wire [7:0] imm;
wire [1:0] opcode;
wire [15:0] pcO;
wire shldBranch;
wire enfetch;
wire endec;
wire enalu;
wire enmem;
wire enrgrd;
wire enrgwr;
wire [15:0] inst;
assign inst = dataO;
assign opcode = (reset) ? 2'b11 :
    (shldBranch) ? 2'b10 :
    (regwe) ? 2'b01 :
    2'b00;

register_file main_reg(
    .I_clk(clk),
    .I_en(enrgrd),
    .I_we(enrgwr),
    .I_selA(selA),
    .I_selB(selB),
    .I_selD(selD),
    .I_dataD(aluData),
    .O_dataA(dataA),
    .O_dataB(dataB)
);

processor16 main_decoder(
    .I_clk(clk),
    .I_en(endec),
    .I_inst(inst),

```

```

        .O_aluop(aluop),
        .O_selA(selA),
        .O_selB(selB),
        .O_selD(selD),
        .O_imm(imm),
        .O_regwe(regwe)
    );
alu main_alu(
    .I_clk(clk),
    .I_en(enalu),
    .I_aluop(aluop),
    .I_dataA(dataA),
    .I_dataB(dataB),
    .I_imm(imm),
    .O_dataResult(aluData),
    .O_shldBranch(shldBranch)
);
ctrl_unit main_ctrl(
    .I_clk(clk),
    .I_reset(reset),
    .O_enfetch(enfetch),
    .O_endec(endec),
    .O_enrgrd(enrgrd),
    .O_enalu(enalu),
    .O_engwr(engwr),
    .O_enmem(enmem)
);
pc_unit main_pc(
    .I_clk(clk),
    .I_opcode(opcode),
    .I_pc(aluData),
    .O_pc(pcO)

```

```

);
fake_ram main_ram(
    .I_clk(clk),
    .I_we(enmem),
    .I_addr(pcO),
    .I_data(aluData),
    .O_data(dataO)
);
initial begin
    clk = 0;
    reset = 1;
    #20;
    reset = 0;
    #500;
    $stop;
end
always #5 clk = ~clk;
endmodule

```

5. TESTBENCH IMPLEMENTATION

5.1 Register File Testbench (register_unittest.v)

Purpose:

To verify read and write operations of the register file.

Test Cases:

- Write FFFF to R0
- Write 3333 to R2
- Write 4444 to R4
- Read and verify stored values

Results:

All register operations were successfully verified.

Verilog Code:

```
`timescale 1ns / 1ps
```

```

module register_unitest;

    reg      I_clk;
    reg      I_en;
    reg      I_we;
    reg [2:0] I_selA;
    reg [2:0] I_selB;
    reg [2:0] I_selD;
    reg [15:0] I_dataD;
    wire [15:0] O_dataA;
    wire [15:0] O_dataB;
    register_file DUT(
        .I_clk(I_clk),
        .I_en(I_en),
        .I_we(I_we),
        .I_selA(I_selA),
        .I_selB(I_selB),
        .I_selD(I_selD),
        .I_dataD(I_dataD),
        .O_dataA(O_dataA),
        .O_dataB(O_dataB)
    );

    /* Test Sequence

        R0 = FFFF
        R2 = 3333
        R4 = 4444

    */ initial
begin
    I_clk  = 0;
    I_en   = 0;
    I_we   = 0;
    I_selA = 0;
    I_selB = 0;

```

```
I_selD = 0;
I_dataD = 16'h0000;
#5;
I_en = 1;
// Write FFFF to R0
#5;
I_selD = 3'd0;
I_dataD = 16'hFFFF;
I_we = 1;
#10;
I_we = 0;
// Write 2222 to R2
#10;
I_selD = 3'd2;
I_dataD = 16'h2222;
I_we = 1;
#10;
I_dataD = 16'h3333;
#10;
I_we = 0;
// FEED should not be written
#10;
I_selD = 3'd0;
I_dataD = 16'hFEED;
// Write 4444 to R4
#10;
I_selD = 3'd4;
I_dataD = 16'h4444;
I_we = 1;
#10;
I_we = 0;
// Read R0 at end
```

```

    #50;

    I_selA = 3'd0;
    I_selB = 3'd0;
    #20;
    $finish;
end
// Clock
always
    #5 I_clk = ~I_clk;
endmodule

```

5.2 Decoder Testbench (decoder_unittest.v)

Purpose:

To verify instruction decoding.

Test Instruction:

0001011100000100

Parameters Verified:

- Opcode
- Register Selection
- Immediate Data
- Register Write Enable

Results:

Decoder outputs matched expected values.

Verilog Code:

```

`timescale 1ns / 1ps
module decoder_unittest;

    // Inputs
    reg I_clk;
    reg I_en;
    reg [15:0] I_inst;

    // Outputs
    wire [4:0] O_aluop;

```

```

wire [3:0] O_selA;
wire [3:0] O_selB;
wire [3:0] O_selD;
wire [7:0] O_imm;
wire O_regwe;
// DUT Instantiation
processor16 DUT (
    .I_clk(I_clk),
    .I_en(I_en),
    .I_inst(I_inst),
    .O_aluop(O_aluop),
    .O_selA(O_selA),
    .O_selB(O_selB),
    .O_selD(O_selD),
    .O_imm(O_imm),
    .O_regwe(O_regwe)
);
// Stimulus
initial
begin
    // Time = 0
    I_clk = 0;
    I_en = 0;
    I_inst = 16'd0;
    // Time = 10
    #10;
    I_inst = 16'b0001011100000100;
    // Time = 20
    #10;
    I_en = 1;
    // Run simulation
    #50;

```

```

    $stop;

end\

// Clock generation (10 ns period)

always

begin

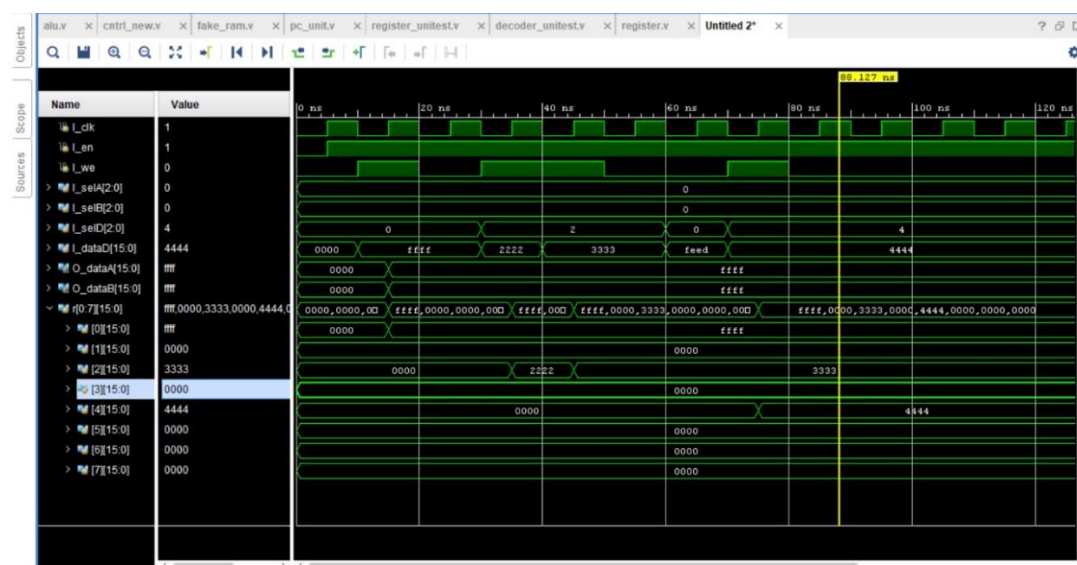
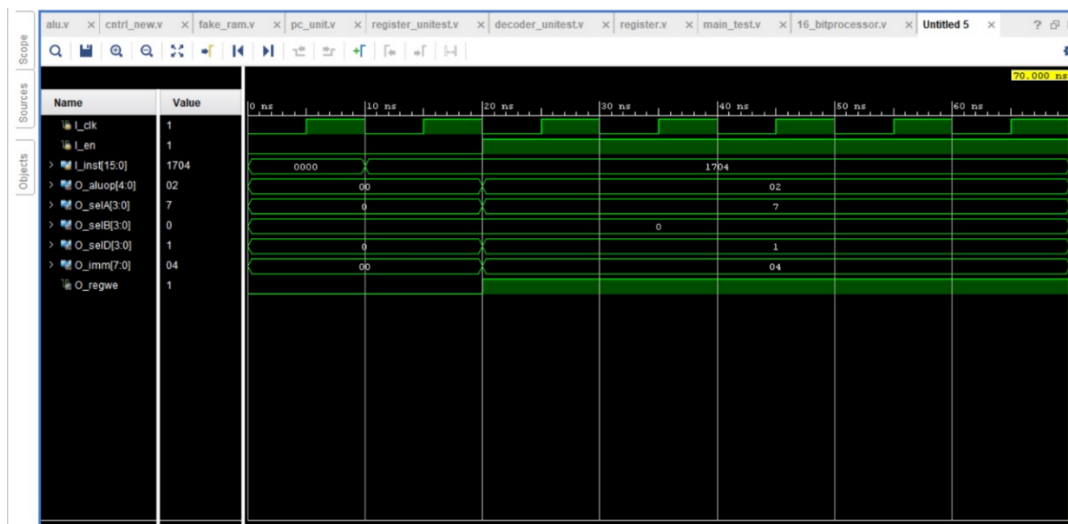
    #5 I_clk = ~I_clk;

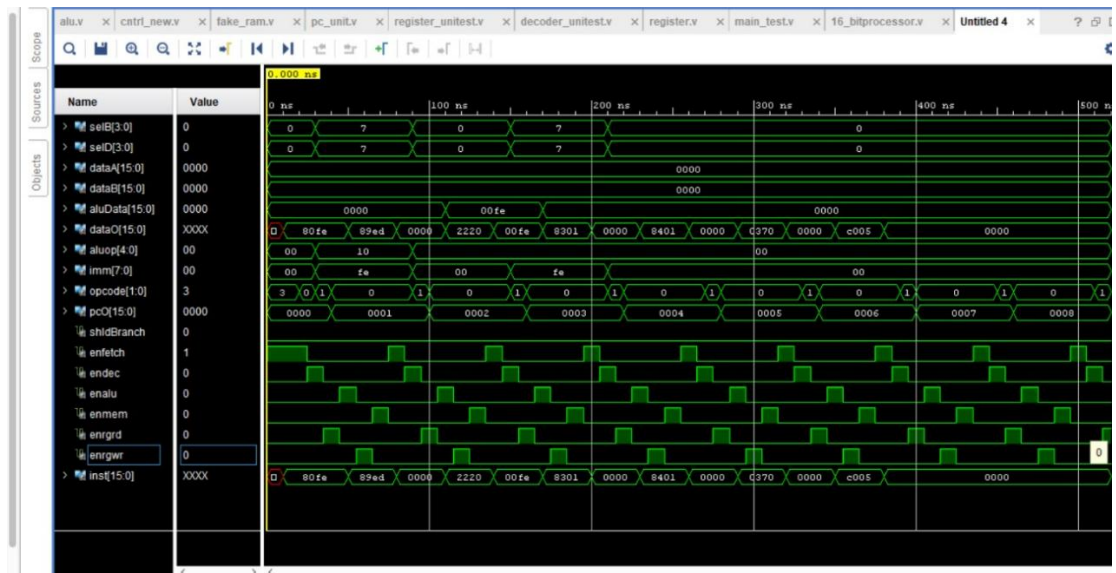
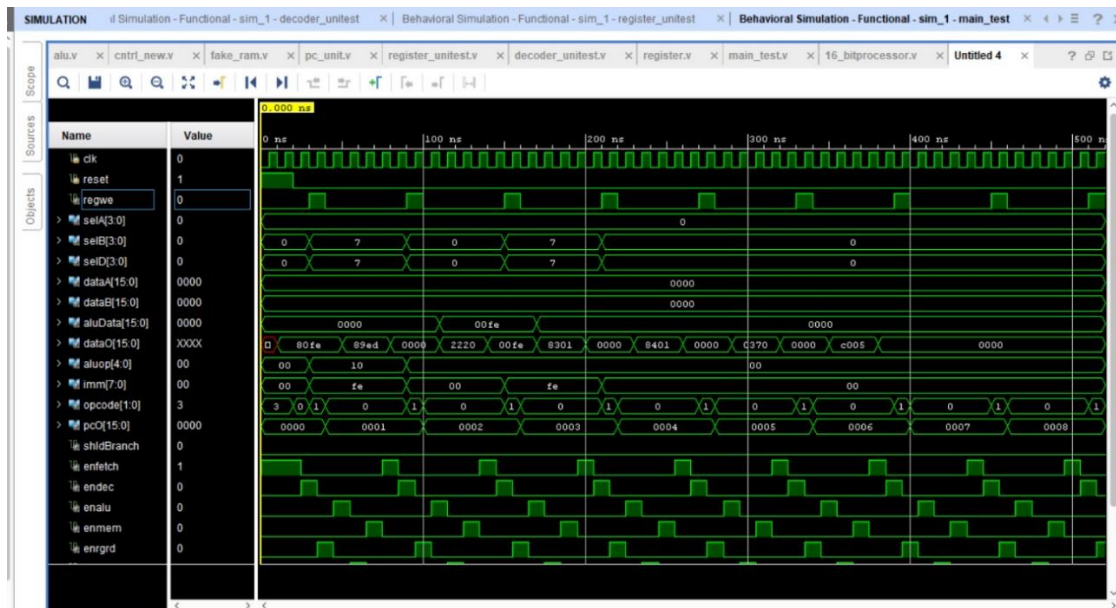
end

endmodule

```

6. SIMULATION RESULTS





7. ADVANTAGES

- Modular Design
- Easy Debugging
- FPGA Compatible
- Expandable Architecture
- Supports Multiple Instructions

8. APPLICATIONS

- Educational Processor Design

- FPGA-Based Systems
- Embedded Systems
- Computer Architecture Learning
- Digital System Prototyping

9. FUTURE ENHANCEMENTS

- Pipelined Processor Design
- Interrupt Handling
- Cache Memory
- UART Communication
- Expanded Instruction Set Architecture

10. CONCLUSION

A complete 16-bit processor was successfully designed and implemented using Verilog HDL. The processor consists of an ALU, Control Unit, Register File, Instruction Decoder, Program Counter, and RAM. Testbenches verified correct operation of all modules. Simulation results confirmed successful execution of arithmetic, logical, memory, and branching instructions. The project provides a strong foundation for understanding processor architecture and digital system design.